Object-Oriented Programming in Java

01 / Trace execution → 02 / Object contracts → 03 / Data and types → 04 / Change and failure → 05 / Application design

01 / Compilation is one check

Compile-time type checks do not prove intended behavior. Trace numeric conversions, handle runtime failures and test the contract, especially integer division, overflow and empty inputs.

02 / Java passes values

A method receives copies of argument values. A copied reference can reach shared mutable state; reassigning the parameter never reassigns the caller’s variable.

03 / Array boundaries

For a non-null array, valid indices satisfy $0 \le i <$ length. A multidimensional array can have null or unequal-length rows; an outer clone is shallow.

04 / Identity and equality

Use == for reference identity and enum constants; use equals for value equality when the class defines it. Equal objects must have equal hash codes; the reverse implication is false.

05 / Preserve invariants

Construct valid objects and check each operation before invalid state escapes. private and final help, but neither automatically makes a referenced object immutable.

06 / Two method-selection stages

Compile-time argument types select an overload. The runtime receiver chooses an overridden instance implementation. Static methods hide, and changing only a return type does not create an overload.

07 / Choose by collection contract

List preserves order and duplicates, Set models uniqueness, and Map associates keys with values. Compare lookup, insertion and traversal costs rather than assuming one implementation is universally faster.

08 / Generic data flow

Generics are invariant. Use ? extends T for a source of T and ? super T for a destination of T. These are type guarantees, not complete immutability guarantees.

09 / Failure and cleanup

Checked exceptions must be caught or declared. Catch where recovery is meaningful, retain the cause, and use try-with-resources for owned resources. Garbage collection does not replace timely resource cleanup.

10 / Tests and recursion

Test normal cases, boundaries and rejection state. Recursive code also needs a base case and decreasing problem size; memoization avoids repeated states but may leave deep call stacks.

11 / Lambda context

A lambda needs a functional-interface target and captures final or effectively final locals. Captured objects may remain mutable; lambda this refers to the enclosing instance.

12 / Design and diagnose

Separate model rules, event input and rendering. Reproduce defects, inspect the earliest divergence, test a specific hypothesis, and document accepted inputs, effects, outputs and failure behavior.